



HOW AI WILL CHANGE SOFTWARE ENGINEERING

– WITH MARTIN FOWLER

Elena Quintes García – UO269665

Sergio Fernández-Miranda Longo - UO302282

Jaime Alonso Fernández - UO294024



INDEX

01

EXECUTIVE
SUMMARY

02

THREE ERAS OF
ABSTRACTION

03

THE CURATOR
ROLE

04

QUALITY AND
SPEED

05

VIBE
CODING

06

AGILE
EVOLUTION

07

PROFESSIONAL
FUTURE

08

REFERENCES



"Simplicity and clarity are the best defenses against complexity."

"If you are not looking at the output, you are not learning. And you cannot shortcut the learning process."

"High internal quality leads to faster delivery of new features. It is a strictly economic argument."

"Whenever someone says 'it depends' and looks at the trade-offs, my confidence in them increases."

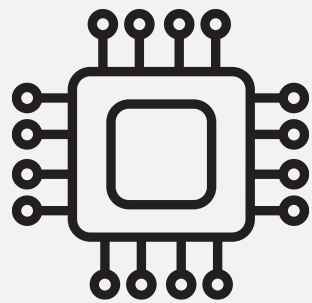
**-MARTIN
FOWLER**



EXECUTIVE SUMMARY

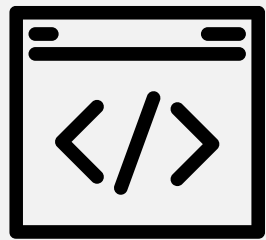
The software industry is facing its most significant transformation since the move from Assembly language to high-level programming.

While AI accelerates output, it introduces non-determinism, making human oversight, testing, and the "learning loop" more critical than ever.



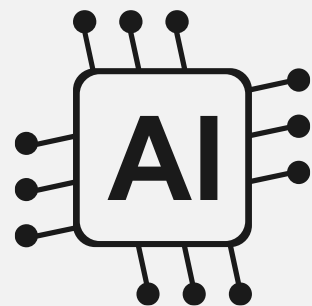
ASSEMBLY ERA

Direct hardware control.
Deterministic, explicit, fully
traceable; but brutally low-level.



HIGH-LEVEL LANGUAGES

Abstraction of memory and
execution.
Reproducible, testable, compiler-
enforced correctness.



AI ASSISTED CODE

Non-deterministic generation.
Fast, powerful; must be
understandable, not just effective.

THREE ERAS OF ABSTRACTION

Fowler describes AI as a
"productive but dodgy
collaborator" → it can
generate a vast amount of
work, but it frequently
"gaslights" the user or
hallucinate errors even in
simple tasks.

THE CURATOR ROLE

ROLE SHIFT

Transitioning from a code "writer" to a "curator" and "reviewer"



VALIDATION IS KEY

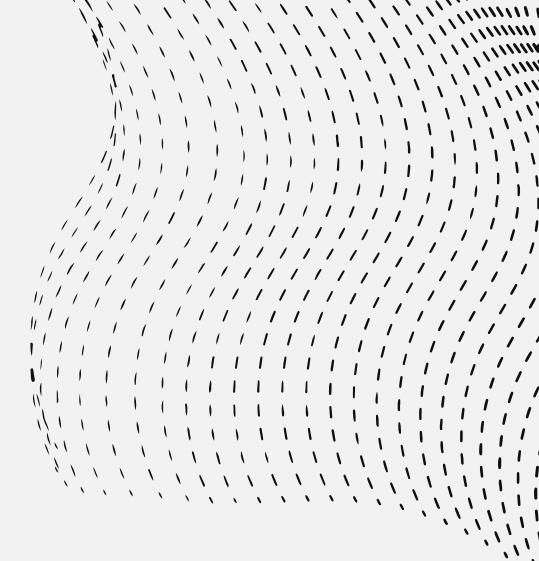
If an engineer cannot validate AI output, they are accumulating unmanaged risk instead of building a system



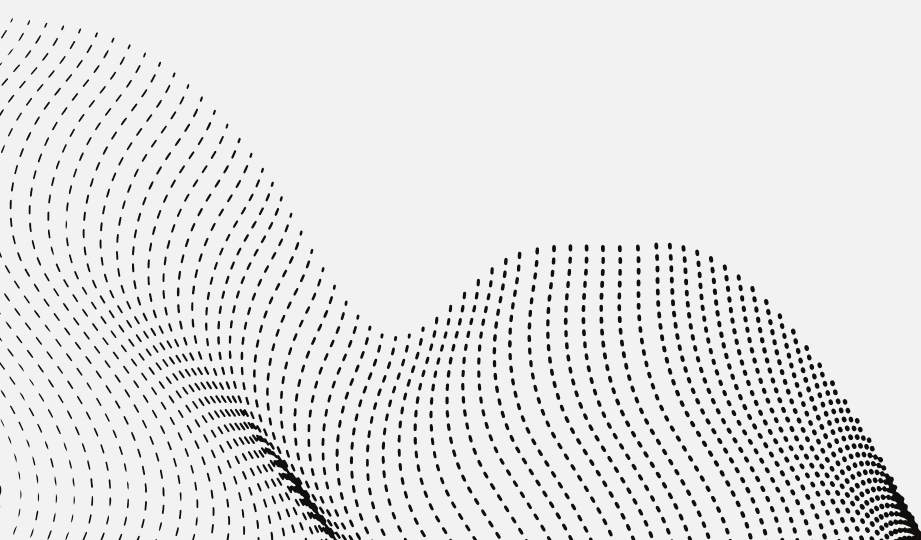
RESPONSIBILITY

The engineer remains responsible for architectural judgment and safety.





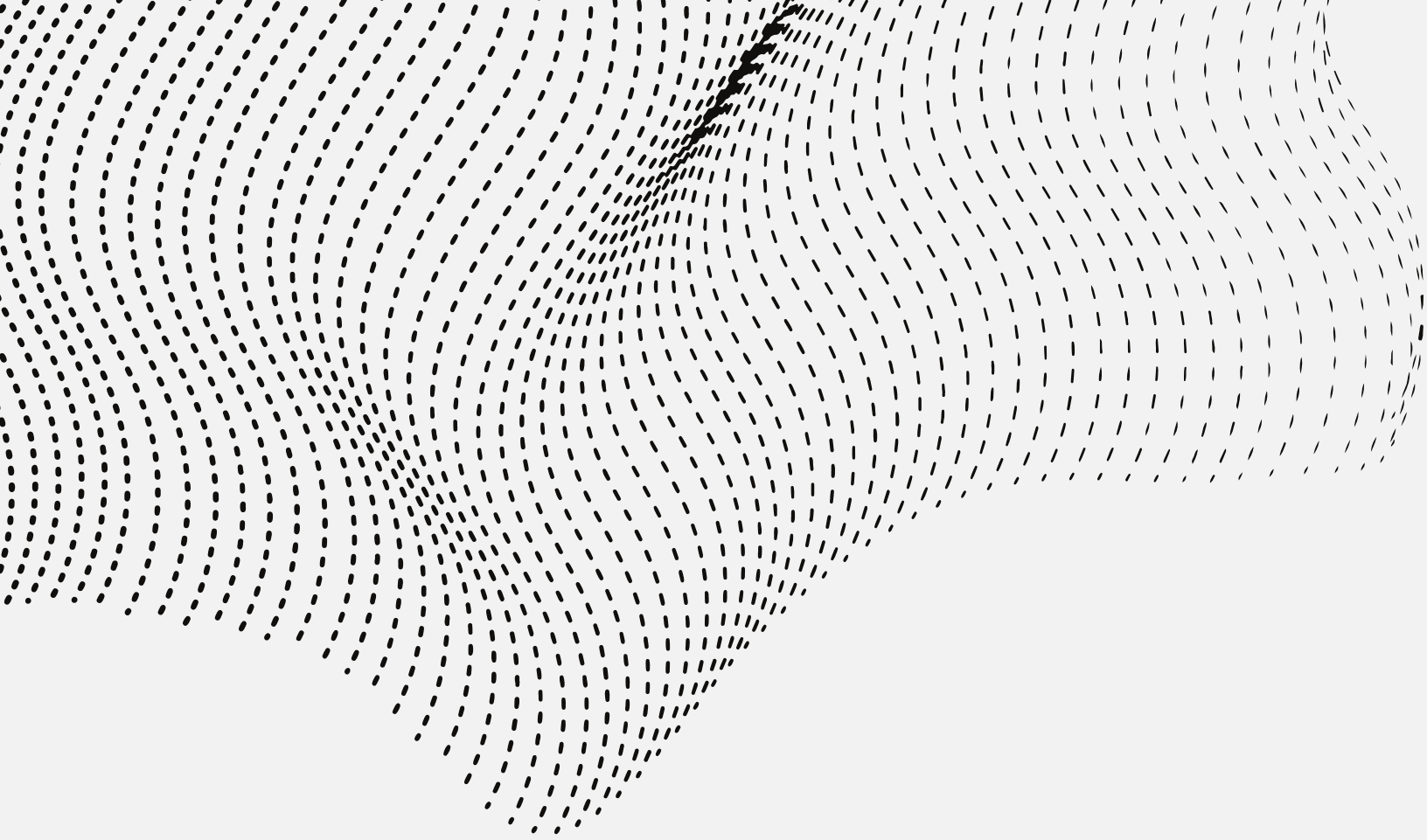
THE ECONOMICS OF QUALITY AND SPEED



A disciplined approach to AI makes development faster.

AI can produce vast amounts of code. If this code is of questionable quality and isn't managed through constant refactoring, the project will eventually collapse under its own complexity.

With AI, refactoring becomes the primary way to ensure generated code remains understandable for humans.



VIBE CODING

Generating code without looking at it or understanding it

SHORTCUT DANGERS

You cannot shortcut the learning process. If a developer doesn't understand the code the AI produces, they cannot evolve it later.

NUKE FROM ORBIT

When you don't understand your code, your only option when it fails is to start over. This is the opposite of sustainable engineering.

FEEDBACK LOOPS

AI should be used to build small, functional pieces of software that are immediately tested and understood by the human in the loop.

AGILE EVOLUTION

SPEC-DRIVEN DEVELOPMENT

While some suggest writing massive prompts (specs) for AI, Fowler advocates for "Thin Slices."

CYCLE TIME

The true leverage of AI is not in doing "more stuff" in one go, but in increasing the frequency of the feedback loop.

LEGACY SYSTEMS

One of the most successful current applications of AI is "interrogating" legacy codebases to understand data flows.



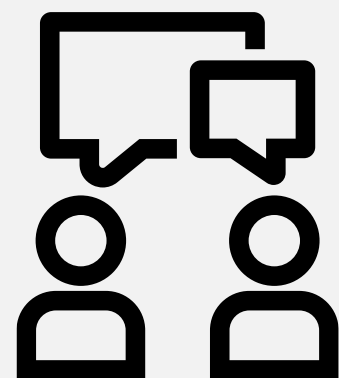
ADVICE FROM JUNIORS

Fowler emphasizes: "Seek a human mentor." A mentor provides context; AI only provides regurgitated data.



THE POWER OF NUANCE

Trust sources that embrace trade-offs. If someone offers answers without nuance, they likely don't understand the complexity of the problem.



CORE SKILLS

The ability to bridge the gap between a business user and a technical system, to understand what needs to be built, is something AI cannot yet replicate.

PROFESSIONAL FUTURE

Fowler remains optimistic but cautious about the "AI Bubble" and the current macroeconomic "depression" in the tech job market.

CONCLUSION

The bottleneck in software has never been the speed of typing code; it has always been the **speed of understanding the problem.**

AI automates the typing, but the engineer remains responsible for the architectural judgment and the safety of the system.

Success in the AI era requires a "verify-first" mindset and a commitment to the craftsmanship of software.



Kent Beck: Mentioned as a peer and mentor whose thinking "leaps forward."



Jim Coplien: Influence on Fowler's early career.



Unmesh Joshi: Author of Patterns of Distributed Systems, cited for his "Expert Generalist" insights.

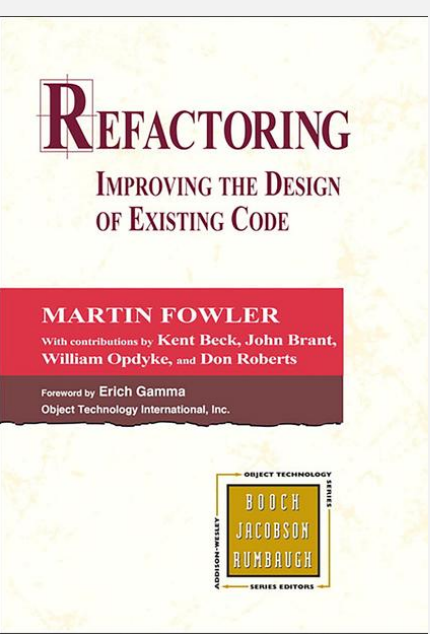
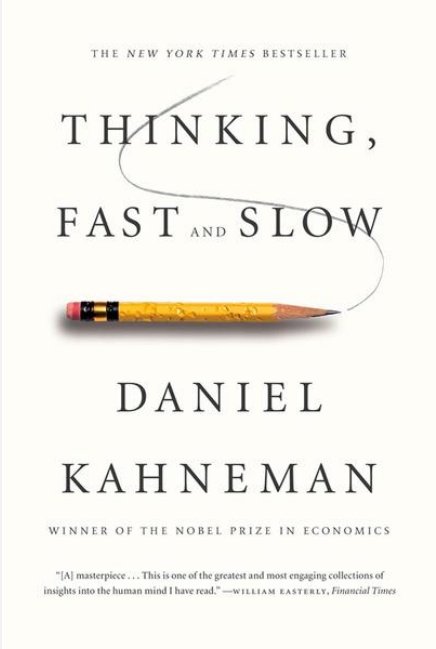


Birgitta Böckeler: Cited as a trusted source of AI engineering knowledge.



Simon Willison: Noted for his high energy and quality output in the AI space.

MENTIONS



Thinking, Fast and Slow (Daniel Kahneman): Recommended for understanding probabilistic reasoning.

The Power Broker (Robert Caro): Recommended for its brilliant writing and analysis of power.

Refactoring (Martin Fowler): His seminal work on small, behavior-preserving changes.

Patterns of Enterprise Application Architecture: Fowler's collection of patterns.

BOOKS



TIME FOR QUESTIONS

– THANK YOU ALL

Elena Quintes García – UO269665

Sergio Fernández-Miranda Longo - UO302282

Jaime Alonso Fernández - UO294024